

Travel Reimbursement Approval Agent

Architecture Note — Generative AI Architect Assignment

Scope: domestic travel reimbursement | Policy TRV-POL v1.0 | Controlled agentic workflow on LangGraph + LangChain

1 Problem Decomposition

A reimbursement claim asks eight questions. They are of different kinds and treating them as one “review this claim” prompt is the usual way these projects fail. The decomposition below is the architecture; the components in section 4 are this table given service boundaries.

Question the claim poses	Nature of the question	Owner	Mechanism
Is the claim well-formed and admissible?	Deterministic — structure, required fields, trip metadata	Claim Intake	Typed schema validation at the API boundary
What do these receipts actually say?	Unstructured — photographs and scans of varied layouts	Receipt Evidence Service	Document/vision extraction with a confidence floor
Which policy clause governs each line?	Semantic — language, not keys	Policy Retrieval Service	Vector search filtered on policy version and effective date
Is the line eligible, and up to what amount?	Arithmetic against limits, caps and per-day rates	Validation / Rule Engine	Deterministic rules, each citing a clause ID
Has this expense been claimed before?	Exact matching over history	Duplicate Check Service	Deterministic match key against the claim’s ledger
Who is entitled to approve this amount?	The ladder is written in the policy; applying it is arithmetic	Approval Matrix Service	Ladder verified against the clause; the lookup itself deterministic
Should a human decide this instead?	Risk composition over the evidence gathered	Safety Gate	Explicit triggers; no model input at this node
How is it explained to the claimant?	Natural language, from a decision already taken	Explanation	Generated from the decision object alone

Order matters as much as ownership. The cheap deterministic checks run first, so a malformed, lapsed or duplicated claim is rejected before it ever reaches a paid extraction call. The explanation runs last, because an explanation written before the decision is a rationalisation.

2 Scope and Business Rules

The first release covers domestic travel and four priced expense categories: air travel (FLT-001, INR 15,000 per one-way sector, economy only), hotel accommodation (HTL-001, INR 8,000 per night inclusive of tax, standard rooms only), meals (MEL-001, INR 1,500 per travel day, alcohol never reimbursable) and local transport (TXI-001, INR 2,000 per travel day, business journeys only). A receipt is required above INR 500 (RCP-001) and at any amount for flights and hotels. A line matching the duplicate key of a settled claim is rejected (DUP-001). Approval authority runs Manager, Senior Manager, Finance Controller by claim value (APR-001).

3 Chosen Workflow and Agent Pattern

3.1 A Controlled agentic workflow

The pattern is an explicit state graph with typed state. LangGraph owns the transitions and the human-in-the-loop interrupt; LangChain supplies the components inside each node — the model client, prompt templates, the policy retriever and the tool bindings. The graph runs eight processing stages across eleven nodes - the three extra are the terminal ones: finalise, manual review and reject-at-intake:

1. Intake and normalise the claim; validate structure and trip metadata.
2. Extract receipt evidence and score extraction confidence.
3. Retrieve the governing policy clauses, filtered to the version in force on the travel dates.
4. Validate each line against its clause; collect exceptions.
5. Check duplicates and resolve the approval tier through deterministic services.
6. Compute approved, rejected and held amounts in the deterministic decision engine.
7. Apply the safety gate: finalise only if every mandatory check passed and the total is within AGT-001.
8. Emit the structured decision with reasons, clause references and audit metadata — or route to Manual Review.

3.2 What the model does, and what it is not allowed to touch

Model is used for	Code owns, exclusively	Containment on the model output
Receipt and document understanding across varied layouts	All arithmetic: applying a limit, multiplying it by nights or sectors, aggregating the buckets	Schema-validated; extracted total cross-checked against the claimed amount
Semantic retrieval of the governing policy clause	Duplicate detection, by deterministic key match against the ledger	Retrieval returns evidence, never authority; an unmatched line is held, not guessed
Classifying ambiguous lines — business travel or sightseeing, food or alcohol	Approval matrix and delegation lookup	Classification can only reduce an approved amount, never increase one
Writing the claimant-facing explanation	Outcome derivation, safety gate and Manual Review routing	Runs after the decision exists; may not introduce a figure not already in it

Three rules make the split enforceable rather than aspirational. A model output may only reduce an approved amount or hold a line - it can never create entitlement. Where a model reports a figure, that figure is one **written in the policy**, and it is verified to appear verbatim in the clause it was read from before it may price anything; an unverified limit prices nothing and the line is held. And the model never performs a calculation: it returns field values, categories and limits as stated, and the engine does the arithmetic. The rule engine imports no model client and opens no socket, which is a property a unit test can assert. Confidence gates autonomy and nothing else - a low-confidence claim is not paid at a discount; it is handed to a person.

3.3 The policy document is the source of the numbers

There is no table of amounts in the codebase. When a claim line arrives, the retriever finds the clause that governs it and the limit is read out of that clause's text - INR 8,000 per night comes from HTL-001, not from a constant. The receipt threshold, the agent's delegation and the whole approval ladder are resolved the same way. The engine holds exactly two numbers of its own, and neither is policy: the extraction confidence floor and the tolerance on a receipt-to-claim mismatch.

4 Main Components

Component	Responsibility	Kind
Claim Intake API	Authenticates the caller, validates the payload against a typed schema, assigns a claim and correlation ID.	Deterministic
Workflow Orchestrator	Owns graph state and transitions, retries idempotent tool calls, records every step, routes exceptions to Manual Review.	Deterministic
Receipt Evidence Service	Extracts merchant, date, amount, currency, receipt ID and category from receipt files; returns values with a calibrated confidence and any claim/receipt mismatch.	Model-backed
Policy Retrieval Service	Indexes the policy corpus by clause; returns the governing clauses with clause ID, source document and effective date.	Model-backed
Validation / Rule Engine	Apply category limits, eligibility and receipt requirements; produces per-line approved, rejected and held amounts, each citing its clause.	Deterministic
Duplicate Check Service	Matches the DUP-001 key against settled and in-flight claims.	Deterministic
Approval Matrix Service	Reads the approval ladder and the agent's delegation from APR-001 and AGT-001, verifies every ceiling and approver title against the clause text, then resolves the tier for a claim total. A ladder that does not verify is discarded and every claim escalates to the highest tier.	Document-sourced, applied deterministically
Decision Engine	Aggregates line results into APPROVE, PARTIALLY_APPROVE or REJECT and composes claim confidence.	Deterministic
Safety Gate	Evaluates the manual-review triggers; permits a finalised decision only when none is open and the total is within AGT-001.	Deterministic
Manual Review Queue	Receives a prepared review packet — exceptions first, then the disallowance schedule; the human outcome returns for audit and evaluation.	Human
Audit Store	Records claim ID, input hashes, retrieved clauses, tool results, state transitions, decision, timestamps and model/prompt/policy versions.	Deterministic

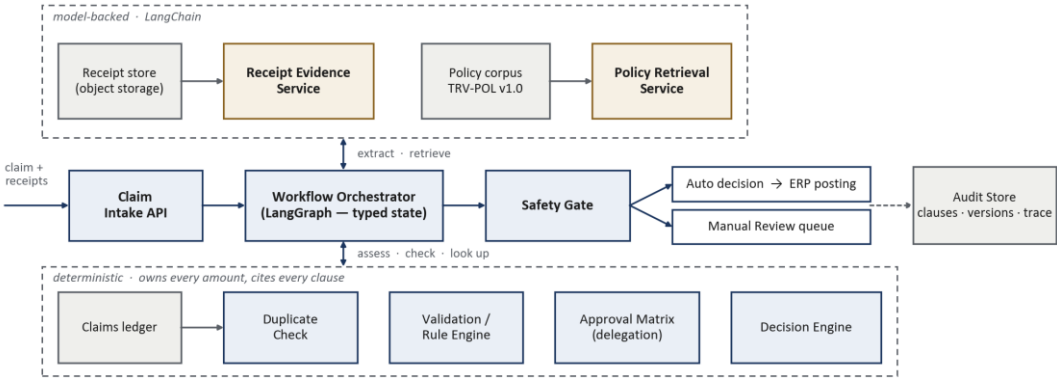


Figure 1. Component architecture. Deterministic services own every amount; the two model-backed services read documents and language only.

5 End-to-End Workflow

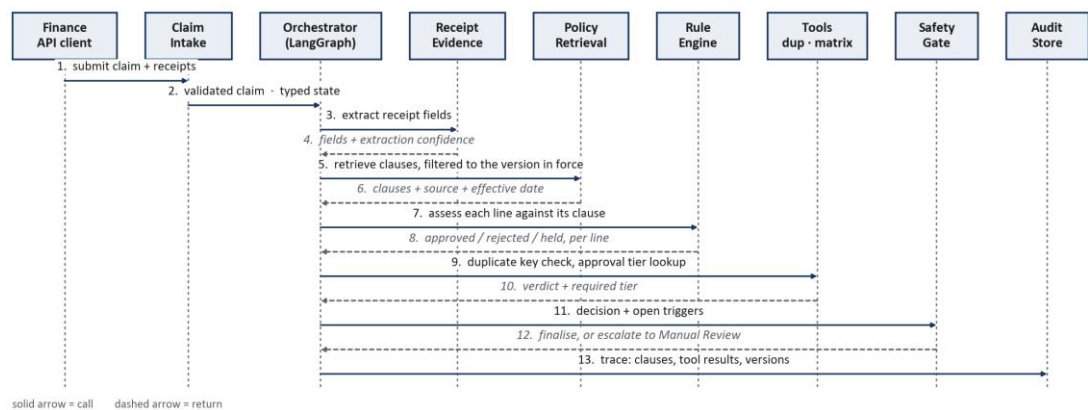


Figure 2. Processing sequence. Eight processing stages, four outcomes; every stage has an explicit failure path and none of them is “approve anyway”.

Stage	What it does	Failure path
1 Intake	Validates the payload against a typed schema and assigns a claim ID	Invalid schema → rejected at intake, before any model call
2 Evidence	Extracts receipt fields with a calibrated confidence	Low confidence, or receipt disagrees with the claim → Manual Review
3 Retrieval	Find the governing clause and read the limit it states	No clause matched, or the limit cannot be verified → Manual Review
4 Validation	Applies the retrieved rule to each line; collects exceptions	Conflicting or incomplete evidence → Manual Review
5 Tools	Duplicate key check and approval-tier lookup	Service failure after retries → Manual Review
6 Decision	Aggregates the three buckets into one outcome	Arithmetic or contract violation → Manual Review
7 Safety gate	Reads open triggers and the delegation, and nothing else	Any trigger open, or total above AGT-001 → Manual Review
8 Audit	Writes the trace, clauses, versions and a reproducible digest	Audit write failure blocks finalisation in production

Two properties carry the reliability argument. Every failure mode resolves to escalation or to a conservative default — there is no path that reads “approve anyway”. And the deterministic gates sit ahead of the two expensive nodes, so a malformed or duplicated claim costs a database read, not an extraction call.

6 Worked Example: One claim, end to end

Everything below is generated by running the reference implementation on claim CLM-002 and capturing what it produced. Nothing here is written by hand, so this section cannot describe behaviour the code does not have. CLM-002 carries three separate policy problems on one claim: a hotel line above the nightly ceiling, alcohol on an itemised meal receipt, and a taxi line with no receipt where one is required.

Input — the claim as submitted.

```
{
  "claim_id": "CLM-002", "employee_id": "E-5290", "employee_name": "Anita Rao",
  "trip_id": "TRP-9014", "trip_from": "2026-08-18", "trip_to": "2026-08-20", "submitted_on": "2026-08-24",
  "lines": [
    { "line_id": "L-1", "category": "FLIGHT", "amount": "6400", "currency": "INR", "description": "Flight Mumbai to Hyderabad, economy",
      "date_from": "2026-08-18", "date_to": "2026-08-18", "sectors": 1, "fare_class": "ECONOMY", "receipt_id": "R-2001" },
    { "line_id": "L-2", "category": "HOTEL", "amount": "9500", "currency": "INR", "description": "Hotel accommodation, 1 night",
      "date_from": "2026-08-18", "date_to": "2026-08-19", "nights": 1, "room_type": "STANDARD", "receipt_id": "R-2002" },
    { "line_id": "L-3", "category": "MEALS", "amount": "4300", "currency": "INR", "description": "Meals during travel",
      "date_from": "2026-08-18", "date_to": "2026-08-20", "travel_days": 3, "receipt_id": "R-2003" },
    { "line_id": "L-4", "category": "TAXI", "amount": "640", "currency": "INR", "description": "Local taxi, hotel to office",
      "date_from": "2026-08-19", "date_to": "2026-08-19", "travel_days": 1, "purpose": "HOTEL_OFFICE", "receipt_id": null }
  ]
}
```

Output — the structured decision. Every reduction names the clause it was made under; the missing document is stated rather than silently rejected.

```
{
  "claim_id": "CLM-002", "employee_id": "E-5290", "currency": "INR",
  "decision": "PARTIALLY_APPROVE", "manual_review": false, "confidence": 0.87,
  "claimed_amount": "20840", "approved_amount": "17450", "rejected_amount": "3390", "held_amount": "0",
  "lines": [
    { "line_id": "L-1", "category": "FLIGHT", "claimed": "6400", "approved": "6400", "rejected": "0", "held": "0",
      "disposition": "APPROVED", "clause": "FLT-001", "reason": "within policy" },
  ]
}
```

```
{
  "line_id": "L-2", "category": "HOTEL", "claimed": "9500", "approved": "8000", "rejected": "1500", "held": "0",
  "disposition": "REDUCED", "clause": "HTL-001", "reason": "above the per night limit of INR 8,000" },
  { "line_id": "L-3", "category": "MEALS", "claimed": "4300", "approved": "3050", "rejected": "1250", "held": "0",
    "disposition": "REDUCED", "clause": "MEL-001", "reason": "non-reimbursable items removed" },
  { "line_id": "L-4", "category": "TAXI", "claimed": "640", "approved": "0", "rejected": "640", "held": "0",
    "disposition": "REJECTED", "clause": "RCP-001", "reason": "receipt missing above the INR 500 threshold" }
},
"missing_documents": [ { "line_id": "L-4", "required": "TAX_INVOICE", "clause": "RCP-001" } ],
"evidence": [ { "clause": "FLT-001", "type": "priced" }, { "clause": "HTL-001", "type": "priced" }, { "clause": "MEL-001", "type": "priced" }, {
"clause": "TXI-001", "type": "priced" } ],
"policy_source": { "document": "TRV-POL v1.0", "effective_from": "2026-04-01" },
"approval": { "tier": "REPORTING_MANAGER", "within_agent_delegation": true, "clause": "APR-001" },
"review_triggers": [ { "code": "RECEIPT_MISSING", "severity": "MEDIUM", "clause": "RCP-001", "line_id": "L-4" } ],
"explanation": "Your claim has been partly approved. Of the INR 20,840.00 claimed, INR 17,450.00 is approved. Reduced: hotel INR
1,500.00 (above the per night limit of INR 8,000, HTL-001); meals INR 1,250.00 (non-reimbursable items removed, MEL-001); taxi INR
640.00 (receipt missing above the INR 500 threshold, RCP-001). Please supply: tax invoice for line L-4.",
"audit": { "correlation_id": "795519e4c605", "policy_version": "TRV-POL v1.0", "workflow_version": "1.0.0", "model_backend": "offline-
deterministic", "trace_digest": "1d546a1cc776b058" }
}
```

The run that produced it — nine nodes, no human required.

```
[1] intake    CLM-002: 4 line(s), Anita Rao, trip TRP-9014 2026-08-18..2026-08-20 [corr 795519e4c605]
[2] evidence  extraction confidence - L-1:0.95, L-2:0.94, L-3:0.92
[3] retrieval L-1 -> FLT-001 (priced, INR 15,000 per_sector); L-2 -> HTL-001 (priced, INR 8,000 per_night); L-3 -> MEL-001 (priced,
INR 1,500 per_travel_day); L-4 -> TXI-001 (priced, INR 2,000 per_travel_day)
[4] validation assessed 4 line(s), approvable INR 17450.00
[5] tools     0 duplicate(s); approver REPORTING_MANAGER
[6] decision  PARTIALLY_APPROVE - claimed 20840, approved 17450.00, rejected 3390.00, held 0
[7] safety_gate finalised within the agent's delegation
[8] finalise  decision final; explanation generated from the decision object
[9] audit     trace written, digest 1d546a1cc776b058
```

All six sample claims, from the same run. Every decision path is exercised, and approved + rejected + held equals claimed on every one.

Claim	Decision	Claimed	Approved	Rejected	Held	Open triggers
CLM-001	APPROVE	24,000	24,000	0	0	none
CLM-002	PARTIALLY_APPROVE	20,840	17,450	3,390	0	receipt missing
CLM-003	REJECT	8,000	0	8,000	0	none
CLM-004	MANUAL_REVIEW	45,100	45,100	0	0	above agent delegation
CLM-005	MANUAL_REVIEW	17,900	5,900	0	12,000	no pricing clause
CLM-006	MANUAL_REVIEW	9,650	450	0	9,200	evidence unreadable, receipt claim mismatch

Produced by the reference implementation - retrieval on tfidf-offline, language tasks on offline-deterministic. Rebuilding this document reruns the claim, so these figures are always the ones the code produces.

7 Tool and Service Contracts

Every interaction that leaves the workflow is a declared contract with a purpose, typed inputs and outputs, a timeout, a retry policy and a stated behaviour on failure. The table is generated from the running catalogue, so it lists what the code actually publishes. Backends here are mocks; in production they are the enterprise document, search, HR and finance services.

Tool	Purpose	Inputs	Outputs	On failure (timeout, retries)
docs.extract_receipt	Turn a receipt image or PDF into structured invoice fields with a calibrated confidence.	receipt_id	merchant, invoice no, date, total, currency, line items, confidence	MANUAL_REVIEW - never estimate an unreadable amount (15000 ms, 1 retry)
controls.check_duplicate	Detect a claim line already submitted or settled within the DUP-001 lookback window.	employee id, claim line, receipt extraction	the matching settled ledger entry, or none	MANUAL_REVIEW - never assume a line is new (1000 ms, 2 retries)
approval.lookup_matrix	Resolve the approver for a claim total under the delegation of authority read from APR-001, and whether it is inside the agent's own delegation under AGT-001.	claim total	approver tier, within_agent_delegation flag	Escalate to the highest tier (700 ms, 2 retries)
policy.search	Retrieve the clauses governing a claim line, filter the policy version in force on the travel	category, line description, travel start date	clause id, type, effective date, similarity score, verified limit	MANUAL_REVIEW - never decide a line without policy

Tool	Purpose	Inputs	Outputs	On failure (timeout, retries)
	dates, and read the limit each states.			evidence (1500 ms, 2 retries)
rules.assess_claim	Compute approved, rejected and held amounts per line from the retrieved rule, and derived the claim outcome.	claim lines, receipt extractions, retrieved rules, receipt threshold	per-line assessment with clause citation, claim totals, open triggers	MANUAL_REVIEW - an arithmetic or contract violation never settles (50 ms, 0 retries)

Read the failure column first. Not one of these resolves to “approve anyway”. Every dependency failure escalates to a person, which is why an outage produces a slow queue rather than a wrong payment - and why `run_demo.py --fail duplicate` turns all six sample claims into Manual Review rather than approving any of them.

8 Assumptions

- Domestic travel only. International travel, per-diem complexity, tax treatment and currency conversion are out of scope; a foreign-currency line raises an exception rather than being converted.
- Policy documents are versioned and carry an effective date. Retrieval is filtered so obsolete and current clauses can never be mixed in one decision.
- Identity, roles and the reporting line are supplied by the enterprise identity provider. The prototype mocks that interface behind the same contract.
- HR, travel-booking, ledger and ERP systems are mocked initially; each sit behind a declared tool contract so a real API can replace it without touching the workflow.
- Manual Review is a valid business outcome and is always preferable to an unsafe automatic decision.
- Model confidence is evidence, never authorisation. It gates autonomy; it never adjusts an amount.

9 Trade-Offs

Decision	Chosen	Alternative	What it costs
Agent pattern	Controlled LangGraph state graph	Free-form autonomous agent	Cannot improvise on unforeseen claim shapes - correct, since those belong with a human.
Policy access	Retrieval, with the limit read from the clause and verified	Limits hard coded in application code	A retrieval layer to operate and tune, in exchange for policy Finance can be edited without a release.
Decisioning	Deterministic engine	Model-generated decision	More engineering; in return, figures that are exact, reproducible and auditable.
Vector store	Local FAISS index for the prototype	Managed enterprise search	No high availability or access control - acceptable for feasibility, not for production.
Human review	An explicit interrupt in the graph	An exception mailbox beside the system	Workflow complexity, in exchange for a governance boundary that can be measured.
Receipt extraction	Model-assisted extraction with a confidence floor	Strict OCR templates	Requires calibration and a held bucket but survives receipt variety.
API shape	Synchronous decision call for the demo	Queue-backed asynchronous workflow	Simple to run and show; throughput and long extractions need the queue in production.

10 Risks, Manual-Review Triggers and Next Steps

Manual Review is entered on any one of these, and the list is deliberately short enough to hold in your head:

- Evidence that cannot be read: extraction confidence below the floor, or a receipt total that disagrees with the claimed amount (EVD-001).
- No priced clause governs the line, or the limit stated in the retrieved clause could not be verified against that clause's own text.
- A line claimed in any currency other than INR, which this policy does not convert (GEN-002).
- A claim total above the agent's delegation under AGT-001.
- A dependency failure that survives its retry policy.
- A claim that fails schema validation at intake - rejected before any model call is made.

Two cases deliberately do not escalate. A line missing a required receipt is rejected under RCP-001 with the document named in `missing_documents`, so the claimant is asked for it rather than a reviewer being asked to wait for it. A line matching the DUP-001 key exactly is rejected outright, because a confirmed duplicate is a fact rather than a judgement. Both keep reviewer attention for the cases that actually need it.

Risk	Mitigation in this design	Next step in production
Prompt injection through receipt text — OCR output is attacker-controlled	The model has no tool access and emits no amount; its structured output feeds a rule engine that text cannot instruct. Worst case is one manipulated field, caught by the total cross-check.	Prompt shields, plus an alert on receipts containing imperative text aimed at a system
Extraction that is confidently	Extracted total is cross-checked against the claim;	Calibrate the floor against reviewer outcomes;

Risk	Mitigation in this design	Next step in production
wrong	anything below the confidence floor is held rather than assessed	sample-audit high-confidence approvals
Policy drift — the engine applies numbers the document no longer states	Every limit carries the clause it came from; policy version is pinned per claim and recorded on the decision	Reconcile limits against clause text as a pipeline gate, with diff-and-approve per policy version
Privacy exposure in receipts and claim data	Encrypted in transit and at rest, role-based access to claims and audit records. The prototype sends receipt text to the model unredacted - acceptable for a feasibility build, not for production	Formal DPIA, retention schedule, and a private model endpoint so no claim data leaves the tenancy

What I would do first in a production build. Assemble three to five hundred historically decided claims and measure agreement against the Finance decision before automating anything — that number, not a demo, is what justifies a delegation limit. Then run in shadow for a full month-end cycle and ship every divergence as a fix. Start the delegation below AGT-001 and raise it on evidence. Instrument cost and latency per claim from day one.

11 Observability, Cost and Latency

A correlation ID is assigned at intake and stamped on every trace step, on the decision and on the audit record. It lives in checkpointed state, so it is the same identifier before and after a claim suspends for human review - one key spans the whole journey, including the part a person did. The audit record holds that ID, the node-by-node trace with timings, the clauses retrieved, the policy and workflow versions, and a digest of the decision. That makes a single claim answerable; running the service needs the aggregate as well.

Measured	Why it is the one to watch
Manual-review rate, split by trigger code	The business KPI. Too high and the agent is not earning its keep; too low and it is deciding things it should not. The split says which evidence or clause is causing it.
Held amount as a share of claimed	Tracks evidence quality and policy coverage directly. A rising held share usually means receipt capture has degraded, or an expense type has appeared that no clause prices.
Extraction confidence distribution, and retrieval scores	Drift here precedes bad decisions. A shift in either is the earliest signal that a model or index change has moved behaviour.
Tokens and cost per claim; tool failure and retry counts	Decides whether the system survives its first budget review, and which dependency is fragile.
Latency per node	Extraction dominates; everything else is noise. Watching the whole claim hides the one slow call.
Reviewer agreement with the recommendation	The only measure of whether the human control is real or decorative, and the number that justifies raising the delegation.

12 Deployment

The logical boundaries do not change between the two cases; only four adapters do - the model endpoint, the document extractor, the search index and the object store, which is precisely why those four are declared tool contracts rather than inline calls. The on-premise build buys data residency and a fixed cost base, and pays for it in GPU capacity planning and weaker extraction, so its confidence floor is set higher and its delegation lower: it degrades into more human review, which is the right direction to fail in.

Concern	On-Premise	Azure, serverless first
Entry point and orchestration	NGINX or the enterprise gateway in the DMZ; Temporal or Argo Workflows on the internal Kubernetes cluster, state in PostgreSQL	API Management with WAF; Durable Functions on a Flex Consumption plan - checkpointed state, fan-out per claim line, scale to zero between month-ends
Inference	vLLM serving Llama 3.1 70B Instruct on 2 x A100 80GB, OpenAI-compatible API so client code is unchanged	Azure OpenAI - gpt-5-mini for extraction, classification and explanation; text-embedding-3-small for retrieval
Documents and retrieval	PaddleOCR or a licensed engine - weaker on poor scans, so the confidence floor is raised; PostgreSQL with pgvector plus a OpenSearch/Elasticsearch	AI Document Intelligence prebuilt receipt model with a page-quality signal; AI Search, BM25 and vector with the semantic ranker, one index alias per policy version
Stores, identity, secrets	MinIO with object lock; PostgreSQL 16 HA under Patroni; Keycloak and HashiCorp Vault; service mesh with mTLS	Blob Storage, Entra ID with managed identities, Key Vault, private endpoints
Human review and operations	A review UI to build and operate; Prometheus, Grafana, Loki and Tempo, plus GPU utilisation	Logic Apps with a Teams adaptive card - approve, amend or reject in place; Application Insights and Monitor workbooks, one trace per claim with token and rupee cost